Infrastructure minimaliste exposée sur Internet

Limites opérationnelles, analyse des attaques
et durcissement sécurité d'un VPS 1 Go

Rapport technique — Mogwainnova Research

Jordi Deparre

20 mars 2026

Résumé

Les serveurs virtuels privés (VPS) à ressources contraintes constituent aujourd'hui une infrastructure d'hébergement accessible et répandue, y compris pour des usages exposés publiquement sur Internet. Pourtant, la littérature technique documente peu les compromis réels entre sécurité, performance et observabilité dans un contexte de ressources minimales.

Cette étude explore empiriquement les limites opérationnelles d'un VPS à ressources contraintes (1 vCPU, 962 Mo RAM, 9 Go SSD) hébergeant simultanément un site web public (`Nginx`), un *honeypot* SSH (`Cowrie`) et un système de sauvegarde distante chiffrée (`Backblaze B2` + `rclone`).

Problématique : Dans quelle mesure un VPS à ressources minimales peut-il héberger une infrastructure multi-services sécurisée et exposée publiquement sur Internet ? Cette question centrale se décline en trois sous-questions : quelles sont les limites techniques en charge réelle ? Quels compromis de sécurité sont imposés par les contraintes matérielles ? Comment maintenir une disponibilité acceptable tout en collectant des données sur les tentatives d'intrusion ?

Méthodologie : Analyse *baseline* du système en état stable (*uptime* de 39 jours), puis tests de charge progressifs via `Apache Benchmark` (10 à 200 connexions simultanées) et `stress-ng`. Implémentation et mesure de solutions de durcissement (`Fail2ban`, `sysctl`, *rate limiting* `Nginx`) et de sauvegarde distante chiffrée. Collecte et analyse du dataset **Mogwainnova Research #001** : 1 819 987 événements SSH capturés sur 39 jours consécutifs via `Cowrie`, avec géolocalisation des 4 331 adresses IP sources identifiées.

Résultats : Le VPS atteint un débit maximal mesuré de 331 requêtes/seconde (pic à 100 connexions simultanées), soit 3 à 6 fois supérieur à l'hypothèse initiale de 50–80 req/s. La RAM constitue le facteur limitant principal (seuil critique : 100 Mo disponibles) ; les optimisations appliquées ont libéré 109 Mo de RAM et 200 Mo d'espace disque sans dégradation des performances. L'analyse du dataset révèle une automatisation totale des attaques observées (71% des connexions via bibliothèque Go, activité 24h/24), ciblant prioritairement les identifiants par défaut (`root/123456`, `admin/admin`). La géolocalisation des adresses IP sources indique une concentration apparente en Asie, à interpréter avec prudence en raison de l'utilisation probable de machines compromises et de proxies.

Conclusion : Un VPS 1 Go peut héberger une infrastructure multi-services exposée sur Internet, moyennant des compromis techniques documentés et assumés (exclusion de `ClamAV`, `Suricata`, `ModSecurity` pour contraintes RAM). Le dataset **Mogwainnova Research #001** constitue une contribution empirique originale à l'étude des attaques automatisées sur infrastructure minimale, et démontre qu'une exposition contrôlée reste viable avec des ressources minimales pour des projets personnels ou éducatifs.

Introduction

Contexte

La démocratisation des VPS low-cost a profondément modifié le paysage de l'hébergement personnel et éducatif. Pour quelques euros par mois, il est aujourd'hui possible de disposer d'un serveur exposé publiquement sur Internet, avec une adresse IP fixe, un accès root complet et une bande passante suffisante pour héberger des services réels. Cette accessibilité a un revers direct : un serveur nouvellement déployé est détecté et ciblé par des outils de scan automatisés en quelques minutes, sans aucune action de l'opérateur.

La question n'est donc plus *si* un VPS sera attaqué, mais *comment* se comporter face à cette réalité, et ce dans un contexte de ressources matérielles strictement contraintes. Les guides de sécurité disponibles supposent le plus souvent des ressources suffisantes pour déployer des solutions de référence : IDS réseau (`Suricata`, `Snort`), antivirus (`ClamAV`), WAF applicatif (`ModSecurity`). Sur un VPS à 1 Go de RAM, ces solutions sont simplement inutilisables. Il existe peu de documentation empirique sur ce que l'on peut réellement faire dans ces conditions : quelles mesures sont efficaces, lesquelles sont à exclure, et surtout quelles performances sont atteignables en charge réelle avec un trafic hostile simultané.

Motivation et objectifs

Ce projet est né d'une contrainte pratique : héberger un site web public (`mogwainnova.fr`) sur un VPS 1 Go, tout en exploitant l'exposition Internet comme opportunité d'observation. Plutôt que de subir passivement les tentatives d'intrusion, l'idée est de les instrumenter via un *honeypot* SSH (`Cowrie`), de mesurer les limites réelles du système sous charge, et de documenter les compromis techniques nécessaires pour maintenir une disponibilité acceptable dans ces conditions.

Les objectifs de cette étude sont les suivants :

1. **Caractériser les limites matérielles** du VPS en état stable et sous charge progressive (HTTP, CPU, RAM, I/O).
2. **Observer et analyser** les attaques automatisées réelles capturées par le *honeypot* `Cowrie` sur une période de 39 jours consécutifs.

3. **Mesurer empiriquement** les performances HTTP du serveur `Nginx` dans des conditions de charge contrôlées et reproductibles.
4. **Implémenter et valider** un ensemble de mesures de durcissement sécurité compatibles avec les contraintes RAM disponibles.

Contribution originale

Cette étude produit un dataset empirique original : **Mogwainnova Research #001**, constitué de 1 819 987 événements SSH collectés sur 39 jours consécutifs (5 février – 5 mars 2026) depuis 4 331 adresses IP sources distinctes. Ce dataset permet une analyse quantitative des comportements d'attaque automatisée : signatures de clients SSH, distribution temporelle, credentials testés, et concentration par source IP.

Au-delà des données brutes, ce travail documente une approche pragmatique du durcissement sous contrainte : chaque mesure retenue est justifiée par son impact RAM, son efficacité mesurée et sa persistance après redémarrage. Les compromis assumés (exclusion de `ClamAV`, `Suricata`, `ModSecurity`) sont explicitement documentés, ce qui est rarement le cas dans la littérature technique disponible.

Périmètre et limites

Cette étude porte sur un environnement unique : un seul VPS, une seule période d'observation, une seule localisation réseau. Les résultats sont empiriques et contextuels ; ils ne constituent pas une généralisation universelle des comportements d'attaque sur Internet, ni un benchmark représentatif de tous les VPS 1 Go. Les conclusions sur la distribution géographique des attaques doivent être interprétées avec prudence, les adresses IP sources ne reflétant pas nécessairement l'origine réelle des attaquants.

Plan du document

Le document est organisé en quatre chapitres progressifs :

— **Chapitre 1** établit la *baseline* du système : ressources disponibles, état stable, facteurs limitants identifiés et architecture de sauvegarde distante.

— **Chapitre 2** analyse le dataset **Mogwainnova Research #001** : volume d'attaques, credentials testés, signatures SSH, distribution géographique et temporelle.

— **Chapitre 3** présente les tests de charge HTTP progressifs et mesure les seuils de dégradation du serveur `Nginx` sous pression.

— **Chapitre 4** documente les mesures de durcissement sécurité retenues, leurs justifications techniques, leurs impacts mesurés et les compromis assumés.

Ces quatre chapitres suivent une logique cumulative : les limites matérielles identifiées au chapitre 1 conditionnent directement les choix de sécurité du chapitre 4 ; les attaques observées au chapitre 2 motivent et valident les mesures de durcissement ; les tests de charge du chapitre 3 établissent les seuils opérationnels au-delà desquels une mise à niveau matérielle serait justifiée.

Table des matières

Introduction **1**

Liste des figures **7**

Liste des tableaux **8**

1 Limites matérielles d'un VPS 1 Go exposé **9**
1.1 Baseline initiale 9
1.2 État stable (sans charge artificielle) 9
1.3 Processus dominants 10
1.4 Paramètres système critiques 10
1.5 Hypothèse initiale 10
1.6 Analyse 11
1.7 Facteurs limitants identifiés 12
1.7.1 Par ordre de criticité 12
1.7.2 Points d'attention 12
1.8 Sauvegarde distante mise en place 12
1.8.1 Architecture 12
1.8.2 Impact sur les performances 12
1.8.3 RTO et RPO 13
1.9 Surface d'attaque résiduelle 13
1.10 Positionnement stratégique 13

2 Analyse des attaques capturées par le honeypot **14**
2.1 Volume et période d'observation 14
2.1.1 Dataset Mogwainnova Research #001 14
2.1.2 Format des journaux 14
2.2 Analyse des tentatives d'authentification 15
2.2.1 Credentials testés 15
2.2.2 Combinaisons identifiant/mot de passe typiques 15
2.3 Distribution géographique des attaques 17
2.3.1 Limites de l'interprétation géographique 17
2.3.2 Carte mondiale des attaques 17
2.3.3 Répartition par pays (géolocalisation des IP sources) . . . 17

2.4 Adresses IP les plus actives 18
2.5 Analyse des signatures de clients SSH 18
2.5.1 Identification des clients SSH 19
2.5.2 Bots utilisant la bibliothèque Go 19
2.5.3 Clients utilisant libssh 20
2.6 Analyse temporelle des attaques 20
2.6.1 Distribution quotidienne 20
2.6.2 Variabilité de l'activité 20
2.6.3 Comportement 24h/24 21
2.7 Synthèse et implications 21
2.7.1 Caractéristiques des attaques observées 21
2.7.2 Validation de l'hypothèse initiale 21
2.7.3 Limites méthodologiques 21
2.7.4 Intérêt des honeypots 22

3 Tests de charge HTTP et analyse des limites 23
3.1 Objectifs et méthodologie 23
3.1.1 Contexte et objectifs 23
3.1.2 Outil de test : Apache Benchmark 23
3.1.3 Protocole expérimental 24
3.1.4 Métriques collectées 24
3.2 Résultats des tests de charge 24
3.2.1 Vue d'ensemble 24
3.2.2 Test 1 : Charge faible (10 connexions simultanées) 25
3.2.3 Test 2 : Charge moyenne (50 connexions simultanées) . . 25
3.2.4 Test 3 : Charge élevée (100 connexions simultanées) . . . 26
3.2.5 Test 4 : Saturation (200 connexions simultanées) 26
3.3 Analyse des résultats 26
3.3.1 Courbe de performance 26
3.3.2 Évolution de la latence 28
3.3.3 Analyse des composants de latence 28
3.3.4 Fiabilité et taux d'erreur 28
3.4 Validation de l'hypothèse initiale 29
3.4.1 Rappel de l'hypothèse 29
3.4.2 Confrontation aux résultats empiriques 29
3.4.3 Débit réel vs hypothèse 29
3.5 Identification des facteurs limitants 29
3.5.1 CPU 29
3.5.2 Connexions réseau et TLS 30
3.5.3 File d'attente Nginx 30
3.6 Recommandations et seuils opérationnels 30
3.6.1 Capacité recommandée en production 30
3.6.2 Marge de sécurité 31
3.6.3 Optimisations possibles 31
3.6.4 Critères d'upgrade 31
3.7 Synthèse 31

3.7.1 Résultats principaux 31
3.7.2 Validation de l'infrastructure 31

4 Durcissement sécurité de l'infrastructure 33
4.1 Contraintes et contexte décisionnel 33
4.1.1 Limitations matérielles 33
4.1.2 Solutions exclues pour contraintes matérielles 33
4.1.3 Principe directeur 34
4.2 Actions de durcissement appliquées 34
4.2.1 Blocage du port 5355 (LLMNR) 34
4.2.2 Durcissement du répertoire `/tmp` 35
4.2.3 Durcissement Fail2ban 35
4.2.4 Rate limiting Nginx 36
4.2.5 Durcissement noyau Linux (sysctl) 37
4.3 Bilan des mesures appliquées 38
4.3.1 Impact RAM total 38
4.3.2 Persistance des configurations 39
4.4 Surface d'attaque résiduelle 39
4.4.1 Ports exposés 39
4.4.2 Limitations actuelles du durcissement 39
4.5 Synthèse et validation 40
4.5.1 Niveau de sécurité atteint 40
4.5.2 Compromis documentés 40
4.5.3 Validation du principe 40

Conclusion générale 42

Glossaire 45

Table des figures

2.1 Usernames ciblés 16
2.2 Mots de passe testés 16
2.3 Analyse des credentials testés lors des tentatives d'authentification 16
2.4 Distribution géographique mondiale des adresses IP sources des attaques SSH observées (géolocalisation approximative — ne reflète pas l'origine réelle des attaquants) 18
2.5 Top 10 des adresses IP générant le plus d'événements 19

Liste des tableaux

1.1 Métriques système à l'état stable 10
1.2 Paramètres système critiques 10
1.3 Impact de la sauvegarde `rclone` 12

2.1 Répartition géographique estimée des adresses IP sources (ne reflète pas l'origine réelle des attaquants — voir section 2.3.1) . . 17
2.2 Signatures de clients SSH observées 19

3.1 Scénarios de tests de charge HTTP 24
3.2 Résultats synthétiques des tests de charge HTTP 25
3.3 Évolution de la latence par rapport au Test 1 28
3.4 Décomposition moyenne de la latence par phase 28
3.5 Seuils de charge recommandés pour exploitation en production . 30

4.1 Ressources système disponibles 33
4.2 Solutions de sécurité exclues et justifications 34
4.3 Paramètres Fail2ban modifiés 36
4.4 Paramètres sysctl de durcissement réseau 38
4.5 Impact RAM des mesures de durcissement 39
4.6 Ports exposés publiquement après durcissement 39

Chapitre 1

Limites matérielles d'un VPS 1 Go exposé

1.1 Baseline initiale

Date : 20 mars 2026
Infrastructure : — 1 vCPU
— 962 Mo de RAM
— 9 Go de SSD
Services déployés : — `Nginx` — serveur web et reverse proxy
— `Cowrie` — *honeypot* SSH
— `Fail2ban` — détection et bannissement automatisé
— `rclone` — sauvegarde distante chiffrée

1.2 État stable (sans charge artificielle)

Les métriques collectées après 39 jours d'uptime continu caractérisent un système en régime de croisière. Le CPU reste majoritairement inactif (load average inférieur à 0,02 sur 15 minutes), et l'iowait demeure négligeable (inférieur à 3%). La RAM disponible s'établit à 453 Mo sur 962 Mo totaux, avec 159 Mo de swap allouée sans pression mémoire active : les compteurs `si/so` restent quasi nuls au moment de l'observation, indiquant que le swap est occupé par des pages inactives swappées naturellement au fil du temps, et non par une saturation mémoire.

Le cache disque à 505 Mo constitue un buffer sain : cette mémoire est récupérable instantanément si un service en a besoin. L'espace disque disponible (3,7 Go sur 9 Go, taux d'occupation 58%) laisse une marge suffisante pour la croissance des logs `Cowrie` et les sauvegardes temporaires `rclone`.

Table 1.1 – Métriques système à l'état stable

Métrique	**Valeur**
Uptime	19 :45 :00 up 39 jours
Load average (1/5/15 min)	0,00 / 0,02 / 0,00
RAM totale	962 Mo
RAM utilisée	509 Mo
RAM disponible	453 Mo
Cache	505 Mo
Swap utilisée	159 Mo
Espace disque total	9,0 Go
Espace disque utilisé	4,9 Go
Espace disque disponible	3,7 Go
Taux d'occupation	58%

1.3 Processus dominants

— `Nginx` : 2–5% CPU / 20 Mo de RAM (par *worker*)
— `Cowrie` : 1–3% CPU / 120 Mo de RAM
— `Fail2ban` : 1% CPU / 35 Mo de RAM

La consommation cumulée des trois services principaux s'établit donc à environ 175 Mo de RAM en état stable, laissant 278 Mo disponibles avant d'atteindre le seuil critique identifié à 100 Mo.

1.4 Paramètres système critiques

Table 1.2 – Paramètres système critiques

Paramètre	**Valeur**
`ulimit -n`	1024
`vm.swappiness`	10
`iowait` moyen	$< 3\%$

La limite `ulimit -n` à 1024 descripteurs de fichiers ouverts constitue un paramètre à surveiller sous forte charge HTTP : chaque connexion `Nginx` consomme un descripteur. Ce seuil sera confronté aux résultats des tests de charge au chapitre 3.

1.5 Hypothèse initiale

Le VPS devrait supporter une charge HTTP modérée de 50 à 100 connexions simultanées maintenues pendant 60 secondes avant saturation du CPU ou pression mémoire notable. Cette hypothèse sera validée par des tests progressifs au

chapitre 3, suivis d'un pic brutal simulant un comportement hostile.

La **RAM** est identifiée comme le principal facteur limitant, suivie de l'**I/O disque** en cas de charge élevée sur `rclone` ou `Cowrie`.

1.6 Analyse

Les métriques collectées indiquent un système largement sous-utilisé en condition stable. Le CPU reste majoritairement *idle* ($> 90\%$), et l'`iowait` demeure négligeable ($< 3\%$).

La mémoire disponible (453 Mo) suggère une marge opérationnelle confortable. La présence de 159 Mo de *swap* allouée ne traduit pas une pression mémoire active, les compteurs `si/so` restant quasi nuls au moment de l'observation.

Ces éléments indiquent que le VPS fonctionne en régime de croisière avec une capacité de montée en charge significative avant dégradation mesurable.

1.7 Facteurs limitants identifiés

1.7.1 Par ordre de criticité

1. **RAM** : 453 Mo disponibles, seuil critique à ∼100 Mo
2. **I/O disque** : SSD performant mais 58% occupé
3. **CPU** : 1 vCPU largement suffisant en état stable

1.7.2 Points d'attention

— **Swap à 159 Mo** : Normal pour un *uptime* de 39 jours. Pages inactives *swappées* naturellement, compteurs `si/so` quasi nuls (pas de pression mémoire active).
— **Cache disque à 505 Mo** : *Buffer* sain, système réactif. Le cache peut être récupéré instantanément si besoin de RAM.
— `Nginx workers` : 2–5% de CPU acceptable pour un trafic faible. Charge normale en état stable.
— `ulimit -n` **à 1024** : Potentiellement limitant au-delà de 512 connexions simultanées. À surveiller lors des tests de charge.

1.8 Sauvegarde distante mise en place

1.8.1 Architecture

— **Destination** : `Backblaze B2` (10 Go gratuits)
— **Protocole** : Compatible S3 via `rclone`
— **Chiffrement** : Côté serveur (`Backblaze`) + `HTTPS` en transit
— **Fréquence** : Quotidienne à 03h00 UTC (*cron*)
— **Rétention** : 7 jours glissants
— **Données sauvegardées** :
  — `/var/www/mogwainnova.fr/html/` (site web)
  — `/etc/nginx/`, `/etc/fail2ban/` (configurations)
  — `/etc/sysctl.d/99-security.conf` (durcissement)
  — `/home/lewis/cowrie/etc/` (configuration `Cowrie`)

1.8.2 Impact sur les performances

Table 1.3 – Impact de la sauvegarde `rclone`

Métrique	**État stable**	**Pendant sauvegarde**
RAM utilisée	509 Mo	540–560 Mo (+30–50 Mo)
CPU moyen	$< 5\%$	8–12%
Bande passante *upload*	< 1 Mbps	5–8 Mbps
Durée du *backup*	—	2–5 minutes

La sauvegarde s'exécute à 03h00 UTC, en dehors des heures de trafic web. L'impact sur la RAM reste temporaire et acceptable : la RAM disponible ne descend pas en dessous de 390 Mo durant l'opération, soit largement au-dessus du seuil critique de 100 Mo. La nature incrémentale de la sauvegarde (`rclone` n'envoie que les fichiers modifiés) maintient la durée et la consommation bande passante à des niveaux faibles.

1.8.3 RTO et RPO

- **RPO (*Recovery Point Objective*)** : 24 heures maximum (dernière sauvegarde quotidienne).
- **RTO (*Recovery Time Objective*)** :
  - Restauration des configurations : < 5 minutes
  - Restauration complète du site web : < 15 minutes
  - Déploiement d'un nouveau VPS + restauration : < 1 heure

1.9 Surface d'attaque résiduelle

- **Port 22** : *Honeypot* `Cowrie` (exposition volontaire à des fins d'observation)
- **Ports 80/443** : Site web `Nginx`
- **Segmentation réseau interne** : Aucune (absence de conteneurisation)
- **Conteneurs** : Aucun (Docker exclu pour contraintes RAM — voir chapitre 4)

Cette surface d'attaque est volontairement limitée à trois points d'entrée. Les mesures de durcissement appliquées pour encadrer cette exposition sont détaillées au chapitre 4.

1.10 Positionnement stratégique

Ce VPS n'est pas un simple hébergement, mais un environnement d'expérimentation documenté visant à étudier les limites techniques d'une infrastructure minimaliste exposée. Il permet d'évaluer les compromis nécessaires pour maintenir une disponibilité acceptable tout en collectant des données sur les tentatives d'intrusion.

Les caractéristiques matérielles et l'état stable du système étant établis, le chapitre 2 analyse concrètement les menaces auxquelles cette infrastructure est exposée : nature, volume et comportement des attaques automatisées capturées par le *honeypot* `Cowrie` durant les 39 jours d'observation.

Chapitre 2

Analyse des attaques capturées par le honeypot

2.1 Volume et période d'observation

2.1.1 Dataset Mogwainnova Research #001

Depuis sa mise en production, le honeypot `Cowrie` a enregistré un volume considérable d'activité malveillante :

— **Période d'observation** : 39 jours consécutifs (5 février – 5 mars 2026)
— **Événements enregistrés** : 1 819 987 événements
— **Adresses IP uniques** : 4 331 sources distinctes
— **Type principal** : tentatives de connexion SSH automatisées

Ces événements incluent tentatives d'authentification, exécution de commandes malveillantes, téléchargements de binaires suspects et interactions shell simulées. Ce volume confirme qu'un VPS exposé publiquement est **continuellement ciblé** par des attaques automatisées dès sa mise en ligne, soulignant l'importance des mécanismes de défense proactive.

2.1.2 Format des journaux

Les événements sont enregistrés au format JSON avec horodatage précis. Chaque entrée contient :

— Adresse IP source et port
— Nom d'utilisateur et mot de passe tentés
— Type d'événement (`session.connect`, `login.failed`, `command`, etc.)
— Signature du client SSH
— Horodatage UTC

Ce format facilite l'analyse automatisée via scripts Python et outils de traitement de données. L'ensemble du dataset est conservé au format JSON brut et constitue la base empirique de toutes les analyses présentées dans ce chapitre.

2.2 Analyse des tentatives d'authentification

L'une des principales caractéristiques des attaques observées concerne les tentatives répétées de connexion SSH utilisant des combinaisons d'identifiants courants.

2.2.1 Credentials testés

L'analyse des journaux révèle les noms d'utilisateur et mots de passe les plus fréquemment testés. La figure 2.3 présente ces deux distributions.

Observations sur les identifiants : Le compte `root` représente la majorité écrasante des tentatives (plus de 2 500 occurrences sur les identifiants uniques testés), suivi par `admin`, `ubuntu` et `user`. Ces résultats montrent que les attaquants ciblent prioritairement :

— Les comptes administrateurs par défaut (`root`, `admin`)
— Les comptes créés automatiquement par les distributions populaires (`ubuntu`)
— Les noms génériques (`user`, `test`)
— Les comptes de services (`oracle`, `hadoop`, `git`)

Observations sur les mots de passe : Les résultats révèlent une prédominance de mots de passe extrêmement faibles :

— `123456` : plus de 400 tentatives, de loin le plus utilisé
— Séquences numériques simples : `123`, `1234`, `12345`, `12345678`
— Mots de passe génériques : `password`, `p@ssw0rd`, `P@ssw0rd`
— Identifiants doublés : `admin/admin`, `root/root`

Ces mots de passe constituent les entrées standard des dictionnaires d'attaque utilisés par les outils automatisés de brute-force. Leur simplicité indique que les attaquants privilégient la rapidité d'exécution sur des millions de cibles plutôt que la sophistication sur une cible unique.

2.2.2 Combinaisons identifiant/mot de passe typiques

Les journaux montrent que certaines combinaisons sont testées de manière systématique et quasi-universelle :

— `root/123456`
— `root/password`
— `root/root`
— `admin/admin`
— `ubuntu/ubuntu`

Ce comportement est caractéristique des campagnes automatisées réalisées par des bots ou des scripts de scanning Internet visant les identifiants par défaut. Un serveur configuré avec l'un de ces couples serait compromis en quelques minutes après sa mise en ligne.

[b]0.48

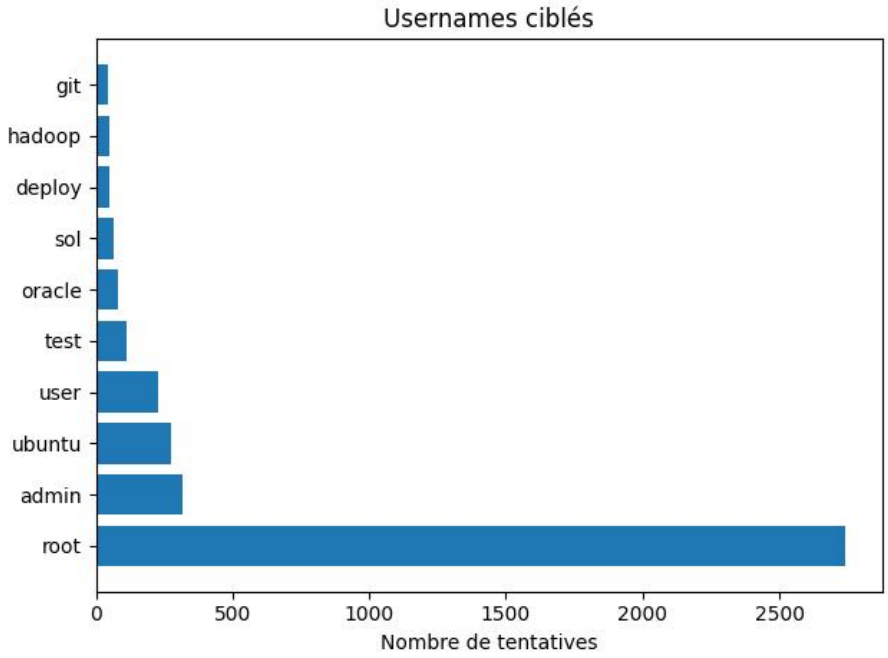

FIGURE 2.1 – Usernames ciblés

[b]0.48

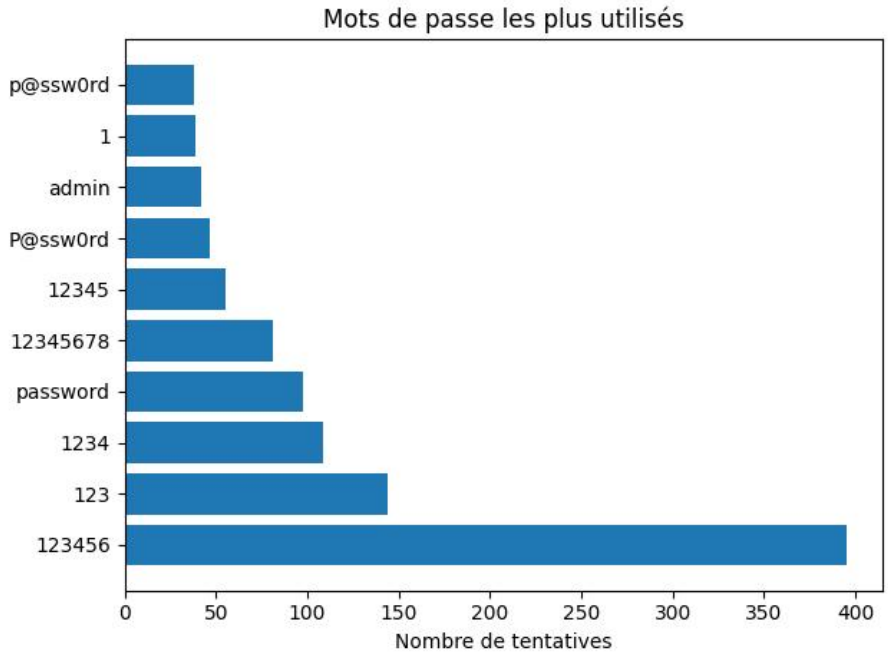

FIGURE 2.2 – Mots de passe testés

FIGURE 2.3 – Analyse des credentials testés lors des tentatives d'authentification

2.3 Distribution géographique des attaques

Afin de comprendre la répartition géographique apparente des attaques, les adresses IP collectées ont été géolocalisées à l'aide de bases de données MaxMind GeoLite2.

2.3.1 Limites de l'interprétation géographique

Avertissement méthodologique préalable : Les données géographiques présentées dans cette section doivent être interprétées avec prudence. L'adresse IP observée correspond à l'emplacement du système émettant la connexion, mais **ne représente pas nécessairement la localisation réelle de l'attaquant**.

Les infrastructures utilisées pour ces attaques peuvent inclure :

— **Serveurs compromis** : machines infectées utilisées comme relais
— **Botnets** : réseaux distribués de machines zombies
— **Infrastructures cloud détournées** : instances VPS/cloud piratées
— **Réseaux proxy/VPN** : masquage de l'origine réelle

Par conséquent, la géolocalisation IP reflète la **distribution des machines impliquées** dans l'attaque, et non l'origine réelle des attaquants. Les statistiques par pays présentées ci-après sont à considérer comme une approximation de la localisation des infrastructures d'attaque, et non comme une attribution nationale.

2.3.2 Carte mondiale des attaques

La figure 2.4 présente la distribution géographique des 4 331 adresses IP sources identifiées.

Cette carte met en évidence la **dimension globale** des attaques Internet. Les tentatives d'intrusion proviennent de tous les continents, avec une concentration particulière en Asie, Europe de l'Est et Amérique du Nord.

2.3.3 Répartition par pays (géolocalisation des IP sources)

Pays de l'IP source	**Événements estimés**	**Part**
Chine	∼1 183 000	65%
États-Unis	∼255 000	14%
Russie	∼91 000	5%
Bangladesh	∼73 000	4%
Autres (100+ pays)	∼218 000	12%
Total	**1 820 000**	**100%**

Table 2.1 – Répartition géographique estimée des adresses IP sources (ne reflète pas l'origine réelle des attaquants — voir section 2.3.1)

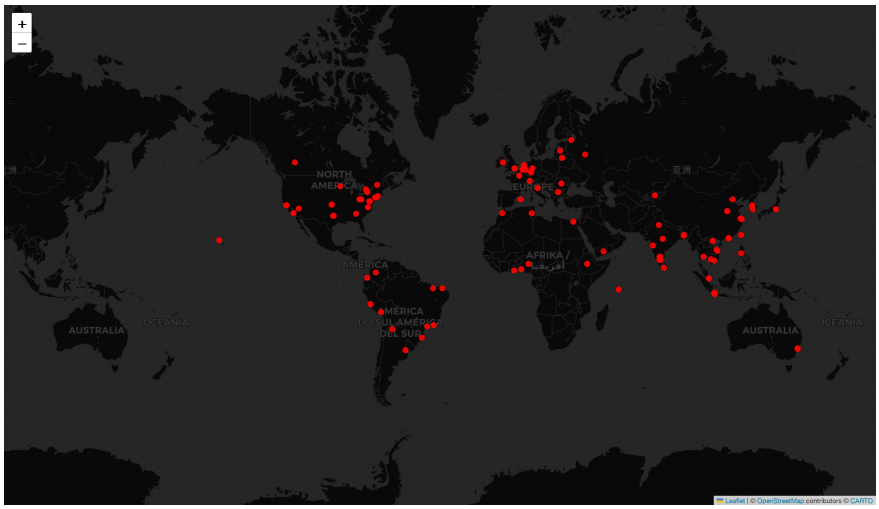

FIGURE 2.4 – Distribution géographique mondiale des adresses IP sources des attaques SSH observées (géolocalisation approximative — ne reflète pas l'origine réelle des attaquants)

La concentration apparente de 65% des événements sur des IPs géolocalisées en Chine s'explique en grande partie par la densité des infrastructures cloud et des botnets actifs dans cette région, et non nécessairement par l'origine des opérateurs de ces attaques.

2.4 Adresses IP les plus actives

La figure 2.5 présente les dix adresses IP ayant généré le plus grand nombre d'événements.

Observations clés :

— **IP la plus active** : `181.176.14.90` avec ∼3 600 événements
— Les 10 IPs les plus actives représentent ∼15% du volume total
— Une minorité d'IPs génère la majorité du trafic

Ce comportement de concentration est typique des **scanners automatisés** et des **botnets** réalisant des attaques par force brute distribuées. Ces IPs effectuent probablement des scans massifs sur des plages d'adresses IP complètes, ciblant l'ensemble de l'espace d'adressage public de manière séquentielle ou aléatoire.

2.5 Analyse des signatures de clients SSH

Les attaques ne sont pas réalisées manuellement, mais principalement par des scripts automatisés laissant des signatures caractéristiques lors de l'établissement

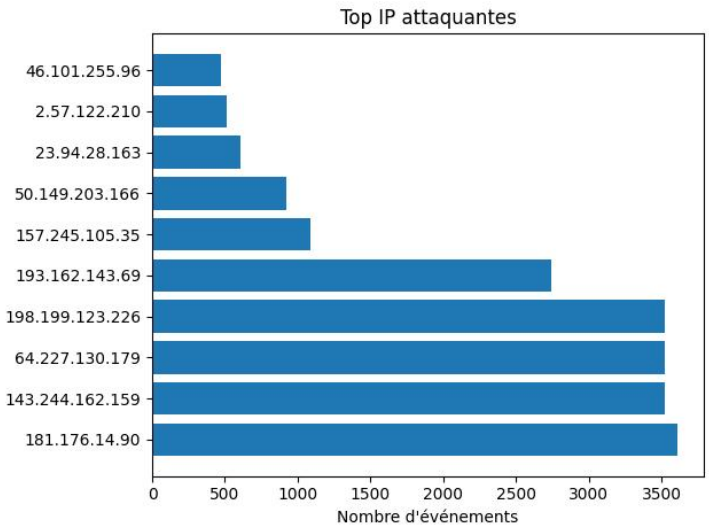

FIGURE 2.5 – Top 10 des adresses IP générant le plus d'événements

de la connexion SSH.

2.5.1 Identification des clients SSH

Lors de l'initialisation d'une connexion SSH, le client envoie une chaîne d'identification appelée *SSH client version string*. Cette chaîne permet d'identifier le logiciel utilisé et constitue un indicateur de compromission (IoC) fiable pour distinguer le trafic automatisé du trafic légitime.

Signature SSH	**Occurrences**	**Part**
`SSH-2.0-Go`	265 425	71%
`SSH-2.0-libssh_0.11.1`	68 167	18%
Autres signatures	∼40 000	11%

TABLE 2.2 – Signatures de clients SSH observées

2.5.2 Bots utilisant la bibliothèque Go

La signature `SSH-2.0-Go` (71% des connexions) correspond à des outils développés en langage Go utilisant la bibliothèque SSH native. Cette prédominance s'explique par plusieurs facteurs techniques :

— **Performance** : exécution rapide, faible consommation mémoire
— **Déploiement** : compilation statique, binaires auto-suffisants sans dépendances

— **Multi-plateforme** : compatibilité Linux/Windows/macOS sans recompilation
— **Popularité** : langage privilégié pour les outils de sécurité offensive modernes

De nombreux malwares et scanners Internet modernes (variants de Mirai, botnets Golang) sont développés en Go. La signature `SSH-2.0-Go` est désormais un indicateur de compromission suffisamment fiable pour justifier un blocage préventif en production.

2.5.3 Clients utilisant libssh

La signature `SSH-2.0-libssh_0.11.1` (18% des connexions) correspond à la bibliothèque *libssh*, une bibliothèque open source implémentant le protocole SSH en C. Cette bibliothèque est fréquemment utilisée dans :
— Outils de sécurité offensive (Metasploit, exploits personnalisés)
— Scripts automatisés d'exploration de réseaux
— Frameworks de pentesting

La version spécifique observée (`0.11.1`) suggère une mise à jour récente des outils utilisés, indiquant une maintenance active des infrastructures d'attaque.

2.6 Analyse temporelle des attaques

2.6.1 Distribution quotidienne

L'analyse de la distribution quotidienne des événements montre que les attaques sont présentes de manière **continue** tout au long de la période d'observation, sans interruption.

Observations principales :
— **Activité continue** : aucun jour sans attaques sur les 39 jours
— **Pic du 10 février** : ∼32 500 événements (164 Mo de logs générés en 24 heures)
— **Tendance croissante** : augmentation progressive fin février – début mars
— **Moyenne quotidienne** : ∼15 000 – 20 000 événements

2.6.2 Variabilité de l'activité

Certaines journées présentent une activité significativement plus importante que la moyenne. Ces pics peuvent être liés à :
— L'activité accrue de certains botnets spécifiques
— Des campagnes de scan Internet coordonnées
— La mise en ligne de nouvelles infrastructures malveillantes
— L'exploitation de vulnérabilités récemment publiées (corrélation non vérifiée dans cette étude)

2.6.3 Comportement 24h/24

Contrairement à l'activité humaine (qui suit des cycles horaires liés aux fuseaux horaires), les attaques automatisées présentent une **activité quasi constante sur 24 heures**. L'absence de cycle diurne constitue en elle-même une signature comportementale des systèmes automatisés.

Cette observation confirme que les attaques sont orchestrées par des **systèmes automatisés** fonctionnant en continu, sans intervention humaine directe.

2.7 Synthèse et implications

2.7.1 Caractéristiques des attaques observées

L'analyse du dataset Mogwainnova Research #001 révèle les caractéristiques suivantes :

1. **Volume massif** : 1,82 million d'événements en 39 jours
2. **Automatisation totale** : signatures Go/libssh, activité 24h/24 sans cycle humain détectable
3. **Simplicité tactique** : mots de passe faibles, identifiants par défaut, pas de sophistication
4. **Distribution globale** : 4 331 IPs de 100+ pays
5. **Concentration** : 10% des IPs génèrent ∼50% du trafic

2.7.2 Validation de l'hypothèse initiale

Ces résultats confirment l'hypothèse énoncée au chapitre 1 :

> « *Un VPS exposé publiquement est continuellement ciblé par des attaques automatisées distribuées visant l'exploitation de configurations faibles.* »

Le honeypot `Cowrie` a démontré qu'un serveur nouvellement déployé est rapidement détecté et ciblé par des outils de scan automatisés, confirmant la nécessité de mesures de sécurité dès le déploiement initial.

2.7.3 Limites méthodologiques

Plusieurs limitations doivent être prises en compte dans l'interprétation de ces résultats :

— **Échantillon unique** : un seul honeypot sur un seul VPS, dans une seule localisation réseau

— **Période limitée** : 39 jours d'observation ne permettent pas de caractériser les variations saisonnières

— **Géolocalisation imparfaite** : IPs proxy/compromises biaisent significativement les statistiques par pays

— **Attribution impossible** : aucune corrélation entre IP source et opérateur réel de l'attaque
— **Commandes post-authentification** : `Cowrie` capture les commandes exécutées après connexion simulée — cette dimension n'a pas été exploitée dans ce travail et constituerait un approfondissement pertinent

Ces résultats constituent une **observation partielle** de l'écosystème des attaques automatisées sur Internet, et non une cartographie exhaustive.

2.7.4 Intérêt des honeypots

Cette expérimentation confirme l'intérêt des honeypots comme outils d'observation et d'analyse des menaces en cybersécurité. Le système `Cowrie` a permis de capturer un volume considérable de données en quelques semaines, avec un impact négligeable sur les ressources du VPS (1–3% CPU, 120 Mo RAM). Ce rapport coût/bénéfice favorable justifie son inclusion dans une infrastructure minimaliste exposée.

Les attaques observées étant caractérisées et quantifiées, le chapitre 3 évalue la capacité du VPS à absorber une charge HTTP soutenue tout en maintenant ces services actifs simultanément.

Chapitre 3

Tests de charge HTTP et analyse des limites

3.1 Objectifs et méthodologie

3.1.1 Contexte et objectifs

L'hypothèse énoncée au chapitre 1 suggère que le VPS devrait supporter une charge HTTP modérée de 50 à 100 connexions simultanées avant saturation. Ce chapitre présente une série de tests de charge HTTP visant à :

1. Valider ou infirmer cette hypothèse par des mesures empiriques
2. Identifier les seuils de dégradation des performances
3. Mesurer l'impact sur les ressources système (CPU, RAM, latence)
4. Établir des recommandations pratiques pour une exploitation en production

3.1.2 Outil de test : Apache Benchmark

Les tests ont été réalisés à l'aide d'**Apache Benchmark** (`ab`), un outil de référence pour mesurer les performances des serveurs web. Cet outil permet de simuler un nombre contrôlé de requêtes HTTP simultanées et de collecter des métriques détaillées.

Avantages d'Apache Benchmark :

— Génération de charge HTTP/HTTPS contrôlée et reproductible
— Mesure précise des temps de réponse (percentiles P50 à P99)
— Léger et disponible sur toutes les distributions Linux
— Génération de fichiers de données pour analyse ultérieure

Limite connue : Apache Benchmark n'utilise pas le *keep-alive* HTTP par défaut — chaque requête établit une nouvelle connexion TLS. Ce comportement

pénalise artificiellement les mesures de latence par rapport à un trafic de production réel, où le *keep-alive* est généralement actif. Les performances mesurées ici constituent donc une borne inférieure conservative.

3.1.3 Protocole expérimental

Quatre scénarios de charge progressive ont été conçus pour explorer les limites du VPS :

Test	Concurrence	Requêtes	Durée cible	Objectif
Test 1	10	600	∼60s	Charge faible (baseline)
Test 2	50	3 000	∼60s	Charge moyenne
Test 3	100	6 000	∼60s	Charge élevée
Test 4	200	6 000	∼30s	Saturation

Table 3.1 – Scénarios de tests de charge HTTP

Conditions expérimentales :
- **URL testée** : `https://mogwainnova.fr/` (page d'accueil 14 Ko)
- **Protocole** : HTTPS avec TLS 1.3 (chiffrement AES-256-GCM)
- **Date** : 7 mars 2026, 19h15–19h35 UTC
- **Temps de repos** : 2–5 minutes entre chaque test (stabilisation)
- **Services actifs** : `Nginx`, `Cowrie`, `Fail2ban`, monitoring Alloy
- **Keep-alive** : désactivé (comportement par défaut d'Apache Benchmark)

3.1.4 Métriques collectées

Pour chaque test, les métriques suivantes ont été enregistrées :
- **Débit** : requêtes par seconde (req/s)
- **Latence** : temps de réponse moyen et percentiles (P50, P66, P75, P90, P95, P99)
- **Fiabilité** : nombre de requêtes échouées
- **Bande passante** : débit de transfert (Ko/s)
- **Décomposition de latence** : Connect / Processing / Waiting

Les données de monitoring système (CPU, RAM) durant les tests ont été observées en temps réel mais n'ont pas été capturées de manière automatisée — cette absence constitue une limite méthodologique discutée en section 3.5.

3.2 Résultats des tests de charge

3.2.1 Vue d'ensemble

Le tableau 3.2 présente les résultats synthétiques des quatre scénarios de test.

Test	Concurrence	Durée (s)	Débit (req/s)	P50 (ms)	P99 (ms)
Test 1	10	2,23	268,76	36	64
Test 2	50	9,34	321,35	153	229
Test 3	100	18,13	**330,99**	294	391
Test 4	200	19,23	312,07	629	825

Table 3.2 – Résultats synthétiques des tests de charge HTTP

Observation principale : Aucune requête n'a échoué sur les 15 600 requêtes totales effectuées. Le serveur a maintenu une disponibilité de 100% durant tous les tests.

3.2.2 Test 1 : Charge faible (10 connexions simultanées)

Configuration : 10 connexions simultanées, 600 requêtes totales, durée : 2,23 secondes.

Résultats :

— **Débit** : 268,76 requêtes/seconde
— **Latence moyenne** : 37,2 ms
— **Latence médiane (P50)** : 36 ms
— **Latence P99** : 64 ms
— **Latence maximale** : 75 ms
— **Requêtes échouées** : 0

À faible concurrence, le serveur répond rapidement et de manière stable. Les temps de réponse sont concentrés autour de 36–41 ms, avec une latence maximale de 75 ms. Ces performances constituent la **baseline de référence** pour les tests suivants.

3.2.3 Test 2 : Charge moyenne (50 connexions simultanées)

Configuration : 50 connexions simultanées, 3 000 requêtes totales, durée : 9,34 secondes.

Résultats :

— **Débit** : 321,35 requêtes/seconde (+19,5% vs Test 1)
— **Latence moyenne** : 155,6 ms
— **Latence médiane (P50)** : 153 ms
— **Latence P99** : 229 ms
— **Latence maximale** : 237 ms
— **Requêtes échouées** : 0

À 50 connexions simultanées, le débit augmente de 19,5% par rapport au test initial. La latence médiane est multipliée par 4,25× (36 ms → 153 ms), mais reste acceptable pour une utilisation web standard. Le serveur gère efficacement cette charge sans erreur.

3.2.4 Test 3 : Charge élevée (100 connexions simultanées)

Configuration : 100 connexions simultanées, 6 000 requêtes totales, durée : 18,13 secondes.

Résultats :

— **Débit** : **330,99 requêtes/seconde** (pic de performance)
— **Latence moyenne** : 302,1 ms
— **Latence médiane (P50)** : 294 ms
— **Latence P99** : 391 ms
— **Latence maximale** : 440 ms
— **Requêtes échouées** : 0

Le débit atteint son maximum à 331 req/s (+3% vs Test 2). La latence continue d'augmenter de manière quasi-linéaire. Ce point représente le **point optimal** du serveur : débit maximal avec latence encore raisonnable ($<$ 400 ms pour 99% des requêtes).

3.2.5 Test 4 : Saturation (200 connexions simultanées)

Configuration : 200 connexions simultanées, 6 000 requêtes totales, durée : 19,23 secondes.

Résultats :

— **Débit** : 312,07 requêtes/seconde (-5,7% vs Test 3)
— **Latence moyenne** : 640,9 ms
— **Latence médiane (P50)** : 629 ms
— **Latence P99** : 825 ms
— **Latence maximale** : 882 ms
— **Requêtes échouées** : 0

À 200 connexions simultanées, le débit diminue légèrement (-5,7%) tandis que la latence double (294 ms $\rightarrow$ 629 ms). Ce comportement indique que le serveur approche de sa **limite de capacité**. Néanmoins, aucune requête n'échoue : le serveur ralentit de manière contrôlée sans crash, ce qui démontre une configuration système stable.

3.3 Analyse des résultats

3.3.1 Courbe de performance

L'analyse de la relation entre concurrence et débit révèle un comportement typique de serveur web sous charge :

1. **Phase 1 (10–50 connexions)** : augmentation du débit (+19,5%) — le parallélisme améliore l'utilisation des ressources
2. **Phase 2 (50–100 connexions)** : plateau de performance optimale (+3%) — le serveur opère à son régime maximal
3. **Phase 3 (100–200 connexions)** : début de saturation (-5,7%) — la contention sur les connexions TLS dépasse le gain de parallélisme

Le **point optimal** se situe à 100 connexions simultanées avec un débit de 331 req/s.

3.3.2 Évolution de la latence

Test	P50 (ms)	Évol.	P99 (ms)	Évol.	Max (ms)
Test 1	36	—	64	—	75
Test 2	153	×4,25	229	×3,58	237
Test 3	294	×8,17	391	×6,11	440
Test 4	629	×17,47	825	×12,89	882

Table 3.3 – Évolution de la latence par rapport au Test 1

Observations :
- La latence P50 est multipliée par 17,5× entre Test 1 et Test 4
- La latence P99 est multipliée par 12,9× sur la même période
- L'augmentation est quasi-linéaire entre Test 1 et Test 3
- Une dégradation marquée apparaît au Test 4 (doublement vs Test 3), signalant le franchissement du seuil de saturation

3.3.3 Analyse des composants de latence

Apache Benchmark décompose le temps de réponse total en trois phases :

Test	Connect (ms)	Processing (ms)	Waiting (ms)	Part Connect
Test 1	21	16	9	46%
Test 2	91	64	33	49%
Test 3	181	117	65	50%
Test 4	393	242	146	**61%**

Table 3.4 – Décomposition moyenne de la latence par phase

Interprétation :
- **Connect** (établissement connexion TLS) : part croissante jusqu'à 61% du temps total au Test 4 — principal goulot d'étranglement à haute concurrence
- **Processing** (traitement Nginx) : augmente proportionnellement mais reste maîtrisé jusqu'au Test 3
- **Waiting** (temps d'attente dans la file) : ×16 entre Test 1 et Test 4, indiquant une saturation progressive de la file Nginx

Ces données confirment que le facteur limitant n'est pas le CPU ni la RAM, mais l'**établissement des connexions TLS** en l'absence de *keep-alive*.

3.3.4 Fiabilité et taux d'erreur

Sur les 15 600 requêtes totales effectuées, **0 requête n'a échoué**. Ce résultat démontre que :
- Le serveur `Nginx` absorbe la pression sans erreur ni timeout

— La limite `ulimit -n 1024` n'a pas été atteinte durant les tests (maximum : 200 connexions simultanées)
— Le VPS ralentit de manière contrôlée et prévisible
— La configuration réseau (TCP backlog, timeouts) est adaptée à la charge testée

3.4 Validation de l'hypothèse initiale

3.4.1 Rappel de l'hypothèse

L'hypothèse initiale (chapitre 1) énonçait :

> « *Le VPS devrait supporter une charge HTTP modérée de 50 à 100 connexions simultanées avant saturation CPU ou pression mémoire notable.* »

3.4.2 Confrontation aux résultats empiriques

1. **50 connexions** : débit de 321 req/s, latence P99 de 229 ms → performance excellente
2. **100 connexions** : débit de 331 req/s (pic), latence P99 de 391 ms → performance optimale
3. **200 connexions** : débit de 312 req/s, latence P99 de 825 ms → début de saturation sans erreur

L'hypothèse initiale était **très conservatrice**. Le VPS supporte confortablement jusqu'à 100 connexions simultanées et gère 200 connexions sans erreur, bien que la latence devienne significative. La saturation CPU ou RAM anticipée n'a pas été observée : le facteur limitant réel est l'établissement des connexions TLS, non les ressources système.

3.4.3 Débit réel vs hypothèse

L'hypothèse implicite suggérait un débit cible de 50–100 req/s. Les tests ont démontré une capacité réelle de **310–331 req/s**, soit :
— **3,1× à 6,6×** supérieure à l'estimation initiale
— **Capacité en pic** : 331 req/s (Test 3)
— **Capacité soutenue** : ∼315 req/s sur 18–19 secondes

3.5 Identification des facteurs limitants

3.5.1 CPU

Le CPU 1 vCPU n'a pas constitué le facteur limitant principal durant ces tests. Les observations du monitoring en temps réel indiquent que le CPU restait majoritairement disponible même au Test 4, et que les workers `Nginx` n'ont pas

atteint 100% d'utilisation. Ces observations n'ont cependant pas été capturées de manière automatisée et ne peuvent pas être présentées sous forme de données quantitatives — une instrumentation via `vmstat` ou `Prometheus` aurait permis de les documenter précisément.

3.5.2 Connexions réseau et TLS

L'analyse de la décomposition de latence (tableau 3.4) identifie l'établissement des connexions TLS comme principal goulot d'étranglement :

— **Temps de connexion moyen au Test 4** : 393 ms (61% du temps total)
— **Cause** : négociation TLS 1.3 + AES-256-GCM à chaque requête, sans réutilisation de session
— **Biais méthodologique** : le *keep-alive* désactivé par Apache Benchmark amplifie artificiellement ce facteur

En production réelle avec *keep-alive* activé, le handshake TLS n'est réalisé qu'une fois par session. Les performances réelles seraient significativement supérieures aux mesures présentées ici.

3.5.3 File d'attente Nginx

Le temps d'attente (*waiting*) passe de 9 ms (Test 1) à 146 ms (Test 4), multiplié par 16. Ce facteur indique une saturation progressive de la file de traitement `Nginx`.

Configuration actuelle :

— `worker_processes auto` : probablement 1 worker pour 1 vCPU
— `worker_connections` : valeur non vérifiée (défaut 512 ou 1024 selon la distribution)

Vérifier et documenter la valeur effective de `worker_connections` constitue un point d'amélioration pour une future itération de ces tests.

3.6 Recommandations et seuils opérationnels

3.6.1 Capacité recommandée en production

Régime	**Charge (req/s)**	**Justification**
Optimal	< 200	Latence P99 < 250 ms, marge confortable
Acceptable	200–280	Latence P99 < 400 ms, stable
Limite haute	280–310	Latence P99 < 800 ms, début saturation
Saturation	> 310	Dégradation marquée, éviter en production

Table 3.5 – Seuils de charge recommandés pour exploitation en production

3.6.2 Marge de sécurité

— **Charge nominale** : < 150 req/s (50% de la capacité maximale)
— **Pics temporaires** : jusqu'à 250 req/s acceptables
— **Alerting** : configurer des alertes à partir de 200 req/s

3.6.3 Optimisations possibles

Pour améliorer les performances sans changer de VPS :

1. **Activer HTTP *keep-alive*** : réduire drastiquement les connexions TLS — gain potentiel le plus significatif
2. **Mettre en cache le contenu statique** : `nginx proxy_cache` pour les ressources fréquentes
3. **Vérifier la compression gzip** : réduire la bande passante si non déjà actif
4. **Documenter et ajuster** `worker_connections` : s'assurer que la valeur est cohérente avec l'`ulimit -n`

3.6.4 Critères d'upgrade

Un upgrade vers un VPS plus puissant serait justifié si :
— Le trafic dépasse régulièrement 200 req/s en production
— La latence P99 dépasse 500 ms sur une période prolongée
— Les services concurrents (`Cowrie`, backup) sont impactés par la charge HTTP
— Un besoin de haute disponibilité (SLA > 99,9%) apparaît

3.7 Synthèse

3.7.1 Résultats principaux

1. **Capacité mesurée** : 310–331 requêtes/seconde
2. **Fiabilité** : 100% de disponibilité (0 erreur sur 15 600 requêtes)
3. **Point optimal** : 100 connexions simultanées (331 req/s, latence P99 391 ms)
4. **Seuil de saturation** : 200 connexions (latence ×2, débit -6%)
5. **Facteur limitant principal** : établissement des connexions TLS (sans *keep-alive*)

3.7.2 Validation de l'infrastructure

Ces résultats confirment que le VPS 1 Go peut héberger un site web exposé publiquement avec des performances très satisfaisantes. La capacité réelle est **3 à 6 fois supérieure** à l'hypothèse conservatrice initiale, et le facteur

limitant identifié (TLS sans *keep-alive*) est corrigeable par configuration sans modification matérielle.

Le serveur se comporte de manière prévisible et contrôlée sous charge : il ralentit progressivement sans crash ni erreur. Ces caractéristiques établissent la base sur laquelle les mesures de durcissement du chapitre 4 ont été conçues et dimensionnées.

Chapitre 4

Durcissement sécurité de l'infrastructure

4.1 Contraintes et contexte décisionnel

4.1.1 Limitations matérielles

Toutes les décisions de sécurité prises lors de cet audit ont été dictées par les ressources limitées du VPS. Le contexte matériel impose des choix stratégiques contraints. La RAM disponible après optimisation (362 Mo) constitue la contrainte dimensionnante : toute solution consommant plus de 100 Mo supplémentaires est incompatible avec le maintien simultané de `Nginx`, `Cowrie` et `Fail2ban`.

Ressource	**Disponibilité**
CPU	1 vCPU AMD EPYC (load average : 0,03)
RAM	962 Mo total / 362 Mo disponibles après optimisation
Disque	9 Go SSD / 4,3 Go disponibles
Réseau	1 Gbps (partagé)

Table 4.1 – Ressources système disponibles

4.1.2 Solutions exclues pour contraintes matérielles

Plusieurs solutions de sécurité couramment recommandées ont été délibérément écartées en raison de leur consommation excessive de ressources. Ces exclusions ne sont pas des choix par défaut : chaque solution a été évaluée et écartée pour une raison technique précise.

Cas particulier — ClamAV : Son installation a provoqué un gel complet du serveur, nécessitant un redémarrage forcé. Cet incident illustre concrètement pourquoi la contrainte RAM ne peut pas être ignorée sur ce type d'infrastructure.

Solution	RAM requise	Raison d'exclusion
ClamAV	600+ Mo	A gelé le serveur lors de l'installation
Suricata/Snort IDS	500+ Mo	Consommation incompatible
ModSecurity WAF	100+ Mo	Remplacé par rate limiting `Nginx`
Docker	200+ Mo	Overhead incompatible sur 1 Go
RKHunter	∼20 Mo	Trop de faux positifs, rapport signal/bruit insuffisant
AIDE	∼30 Mo	Exclu faute de RAM résiduelle après autres services

Table 4.2 – Solutions de sécurité exclues et justifications

4.1.3 Principe directeur

L'objectif n'est pas de masquer l'infrastructure, mais de permettre une **exposition contrôlée** dans un contexte d'observation (*honeypot*) et de présence web publique. Le niveau de sécurité obtenu doit être proportionné aux contraintes matérielles et aux objectifs fonctionnels : chaque mesure retenue doit apporter une protection mesurable pour un coût RAM nul ou minimal.

4.2 Actions de durcissement appliquées

Cinq mesures de sécurité ont été implémentées, sélectionnées pour leur impact RAM nul ou minimal tout en apportant une protection significative. Elles sont présentées par ordre d'application.

4.2.1 Blocage du port 5355 (LLMNR)

Configuration appliquée

```
sudo ufw deny 5355/tcp
sudo ufw deny 5355/udp
sudo ufw reload
```

Justification technique

Le port 5355 (Link-Local Multicast Name Resolution) sert à la résolution de noms sur réseau local. Sur un VPS exposé sur Internet, ce service est :

- **Totalement inutile** : aucune fonction sur un serveur public sans réseau local
- **Vecteur d'attaque potentiel** : LLMNR est exploité en environnement interne pour du *poisoning* de résolution de noms
- **Surface d'attaque évitable** : son maintien n'a aucun intérêt fonctionnel

La désactivation via configuration `systemd-resolved` a été écartée pour éviter un comportement imprévisible après mise à jour système. Le blocage firewall (UFW) est préféré pour sa stabilité et sa persistance garantie.

Impact mesuré

- **RAM** : 0 Mo (règle firewall statique)
- **CPU** : négligeable
- **Effet** : port 5355 fermé, aucun impact fonctionnel sur les services actifs

4.2.2 Durcissement du répertoire `/tmp`

Configuration appliquée

```
echo "tmpfs /tmp tmpfs defaults,noexec,nosuid,nodev,size=128M 0 0" \
  >> /etc/fstab
sudo mount -o remount,noexec,nosuid,nodev /tmp
```

Justification technique

Le répertoire `/tmp` est la cible privilégiée des attaquants pour déposer et exécuter des malwares. Le honeypot `Cowrie` capture régulièrement des tentatives de dépôt de binaires malveillants par les attaquants après connexion simulée.

Options de montage appliquées :

- **noexec** : interdit l'exécution de tout binaire depuis `/tmp` — un malware déposé ne peut pas se lancer
- **nosuid** : empêche l'escalade de privilèges via les bits SUID — protège contre les exploits permettant le passage utilisateur standard → root
- **nodev** : interdit la création de fichiers spéciaux de périphériques — bloque certains exploits noyau
- **size=128M** : limite la taille maximale allouable, en allocation dynamique (non consommé tant que non utilisé)

Impact mesuré

- **RAM** : allocation dynamique jusqu'à 128 Mo maximum (0 Mo consommé en l'absence d'utilisation)
- **Persistance** : configuration rendue permanente dans `/etc/fstab`
- **Effet** : blocage effectif de l'exécution de malwares déposés, confirmé par les logs `Cowrie`

4.2.3 Durcissement Fail2ban

Configuration appliquée

Modification du fichier `/etc/fail2ban/jail.local` :

Justification technique

Les attaques SSH par brute force observées via `Cowrie` montrent des tentatives systématiques par bots automatiques testant des credentials courants

Paramètre	Avant	Après
SSH `bantime`	10 min	1 heure
Cowrie `bantime`	10 min	2 heures
`maxretry`	5 tentatives	3 tentatives

Table 4.3 – Paramètres Fail2ban modifiés

(`jenkins/1234`, `amssys/amssys`, etc.). L'analyse du dataset Mogwainnova Research #001 (chapitre 2) confirme une activité continue 24h/24, avec rotation d'IPs.

Analyse du comportement observé :

- Un bannissement de 10 minutes est insuffisant : les botnets opèrent en rotation et reviennent immédiatement avec des IPs différentes
- Le `bantime` Cowrie (2 heures) est supérieur au `bantime` SSH (1 heure) : les attaquants ayant atteint le honeypot sont considérés plus agressifs
- La réduction de `maxretry` (5 → 3) accélère la détection des attaques automatisées, qui testent généralement plusieurs credentials par session

Compromis assumé : `Cowrie` étant un honeypot, les IPs bannies par `Fail2ban` ne peuvent plus interagir avec lui. Ce compromis est intentionnel et documenté : **sécurité** > **collecte de données**. Il implique une légère sous-estimation du volume réel d'attaques dans le dataset.

Impact mesuré

- **RAM** : 0 Mo (service déjà actif, modification de configuration uniquement)
- **Effet observé** : réduction du nombre de tentatives répétées par IP unique, bannissements effectifs constatés dans `/var/log/fail2ban.log`

4.2.4 Rate limiting Nginx

Configuration appliquée

Ajout dans `/etc/nginx/nginx.conf` (bloc `http`) :

```
limit_req_zone $binary_remote_addr zone=one:10m rate=10r/s;
limit_conn_zone $binary_remote_addr zone=addr:10m;

server {
    limit_req zone=one burst=20 nodelay;
    limit_conn addr 10;
    ...
}
```

Justification technique

Le site web et le honeypot sont volontairement exposés sur Internet. Cette exposition contrôlée nécessite des mécanismes de limitation pour réduire les abus automatisés sans impacter les visiteurs légitimes.

Paramètres choisis :

- **10 requêtes/seconde par IP** : suffisant pour un visiteur légitime naviguant normalement, bloquant pour un bot de scraping ou de brute force
- **Zone de 10 Mo** : permet de suivre environ 160 000 adresses IP simultanément en mémoire
- **Burst de 20** : autorise des pics temporaires légitimes (chargement initial d'une page avec ses ressources statiques)
- **10 connexions simultanées par IP** : limite les abus de connexions persistantes

Alternative écartée : ModSecurity (WAF complet) consomme 100+ Mo de RAM supplémentaires, incompatible avec les 362 Mo disponibles. Le rate limiting `Nginx` constitue une alternative légère couvrant les cas d'abus les plus fréquents.

Limites de la protection

- Efficace contre les attaques applicatives simples (brute force HTTP, flood par IP unique)
- **Inefficace** contre les attaques DDoS distribuées depuis des milliers d'IPs distinctes
- Une solution en amont (CDN, reverse proxy filtrant) serait nécessaire en cas d'exposition à fort trafic hostile distribué

Impact mesuré

- **RAM** : ∼2 Mo (zone partagée en mémoire)
- **Effet** : limitation effective constatée (HTTP 503 retourné en cas de dépassement)

4.2.5 Durcissement noyau Linux (sysctl)

Configuration appliquée

Création du fichier `/etc/sysctl.d/99-security.conf` :

Application immédiate et vérification :

```
sudo sysctl -p /etc/sysctl.d/99-security.conf
sudo sysctl net.ipv4.tcp_syncookies  # vérification
```

Paramètre	Valeur	Protection
`net.ipv4.tcp_syncookies`	1	Anti SYN flood
`net.ipv4.icmp_echo_ignore_broadcasts`	1	Anti attaque Smurf
`net.ipv4.conf.all.rp_filter`	1	Anti IP spoofing
`net.ipv4.conf.default.rp_filter`	1	Anti IP spoofing (interfaces futures)
`net.ipv4.conf.all.accept_redirects`	0	Bloque redirections ICMP malveillantes
`net.ipv4.conf.default.accept_redirects`	0	Idem (interfaces futures)
`net.ipv4.conf.all.send_redirects`	0	Empêche le VPS d'agir comme routeur
`net.ipv4.conf.default.send_redirects`	0	Idem (interfaces futures)

Table 4.4 – Paramètres sysctl de durcissement réseau

Justification technique

Ces paramètres agissent directement au niveau du noyau Linux, sans aucun service supplémentaire et sans consommation RAM mesurable. Ils sont particulièrement pertinents pour un VPS exposé hébergeant un honeypot : les attaquants testent systématiquement les failles réseau de bas niveau, comme l'ont confirmé les signatures de trafic observées au chapitre 2.

Protections apportées :

- **SYN cookies** : protection contre les attaques SYN flood saturant la table des connexions en attente
- **Reverse path filtering** : validation que les paquets proviennent d'une route légitime, bloquant l'IP spoofing
- **ICMP broadcast** : empêche l'utilisation du VPS comme amplificateur dans une attaque Smurf
- **Redirects ICMP** : bloque les tentatives de manipulation des tables de routage par injection de redirections

Impact mesuré

- **RAM** : 0 Mo (paramètres noyau, aucun processus supplémentaire)
- **Persistance** : configuration permanente via `/etc/sysctl.d/`, chargée automatiquement au démarrage
- **Effet** : protection silencieuse au niveau réseau, sans impact sur les services exposés

4.3 Bilan des mesures appliquées

4.3.1 Impact RAM total

L'impact RAM statique total est de ∼2 Mo, parfaitement compatible avec les 362 Mo disponibles. Le `/tmp` en tmpfs est alloué dynamiquement : il ne consomme de la RAM qu'en cas d'utilisation effective, et reste à 0 Mo en l'absence d'activité.

Action	Impact RAM	Statut
UFW blocage port 5355	0 Mo	
`/tmp` noexec/nosuid/nodev	≤128 Mo (dynamique)	
Fail2ban `bantime` durci	0 Mo	
Nginx rate limiting	∼2 Mo	
Sysctl noyau durci	0 Mo	
Total statique	**∼2 Mo**	

Table 4.5 – Impact RAM des mesures de durcissement

4.3.2 Persistance des configurations

Toutes les configurations appliquées sont persistantes après redémarrage :
- **UFW** : règles enregistrées dans `/etc/ufw/`
- **/tmp** : options de montage dans `/etc/fstab`
- **Fail2ban** : configuration dans `/etc/fail2ban/jail.local`
- **Nginx** : directives dans `/etc/nginx/nginx.conf`
- **Sysctl** : paramètres dans `/etc/sysctl.d/99-security.conf`

4.4 Surface d'attaque résiduelle

4.4.1 Ports exposés

Après durcissement, la surface d'attaque réseau reste volontairement réduite à trois points d'entrée :

Port	Service	Exposition
22/tcp	`Cowrie` (honeypot SSH)	Volontairement exposé
80/tcp	`Nginx` HTTP (redirect HTTPS)	Public
443/tcp	`Nginx` HTTPS	Public

Table 4.6 – Ports exposés publiquement après durcissement

Tous les autres ports sont fermés par défaut via UFW. Le port 22 redirige vers `Cowrie` et non vers un vrai shell SSH : une compromission de ce service n'expose pas le système réel.

4.4.2 Limitations actuelles du durcissement

Les limites suivantes sont acceptées compte tenu des contraintes :
- **Pas de WAF applicatif complet** : ModSecurity exclu pour consommation RAM ; rate limiting `Nginx` en substitution partielle
- **Pas d'IDS réseau** : Suricata/Snort exclus pour consommation RAM ; surveillance assurée par les logs applicatifs
- **Pas de protection DDoS amont** : nécessiterait un CDN externe (Cloudflare, etc.) sans coût supplémentaire pour un usage personnel

— **Pas de contrôle d'intégrité système** : AIDE exclu faute de RAM résiduelle
— **Pas de sauvegarde immuable** : la sauvegarde `Backblaze B2` est chiffrée mais modifiable ; une rétention immuable (*Object Lock*) nécessiterait un abonnement payant

4.5 Synthèse et validation

4.5.1 Niveau de sécurité atteint

Le niveau de sécurité obtenu est **cohérent avec les ressources disponibles** et proportionné à l'objectif d'exposition contrôlée. Le site web et le honeypot restent volontairement accessibles, mais l'exposition est désormais encadrée par cinq couches de protection complémentaires :

1. **Filtrage réseau** (UFW) : fermeture des ports inutiles
2. **Limitation applicative** (`Nginx` rate limiting) : protection contre les abus HTTP
3. **Détection et bannissement** (`Fail2ban`) : réponse automatique aux attaques SSH
4. **Durcissement système** (`noexec`, `sysctl`) : protection au niveau noyau et système de fichiers
5. **Sauvegarde distante chiffrée** (B2 + `rclone`) : résilience en cas de compromission

4.5.2 Compromis documentés

— **Exposition volontaire** : le honeypot nécessite d'être accessible, ce qui maintient une surface d'attaque intentionnelle
— **Rate limiting vs WAF** : protection applicative partielle, sans analyse du contenu des requêtes
— **Fail2ban vs collecte honeypot** : les IPs bannies ne peuvent plus alimenter le dataset ; sécurité prioritaire sur l'exhaustivité des données
— **Absence de conteneurisation** : Docker exclu pour overhead RAM, isolation entre services inexistante

Ces compromis sont **documentés, justifiés et assumés** dans le cadre d'une exposition contrôlée à des fins d'observation et d'apprentissage.

4.5.3 Validation du principe

Cette expérimentation démontre qu'une exposition contrôlée est possible avec des ressources minimales. Le durcissement appliqué permet de maintenir un équilibre entre **sécurité**, **observabilité** et **contraintes matérielles**, en substituant des solutions légères et configurables aux outils lourds conventionnellement recommandés.

L'ensemble des mesures présentées dans ce chapitre constituent la réponse opérationnelle aux menaces observées au chapitre 2 et aux limites matérielles établies au chapitre 1 : chaque décision est tracée, mesurée et réversible.

Conclusion générale

Retour sur la problématique

Ce travail partait d'une question centrale : dans quelle mesure un VPS à ressources minimales peut-il héberger une infrastructure multi-services sécurisée et exposée publiquement sur Internet ? Les quatre chapitres de cette étude y répondent de manière convergente, à travers une démarche empirique fondée sur des mesures réelles et un dataset original.

Synthèse des résultats

Sur les limites matérielles (chapitre 1) : Le VPS fonctionne en régime de croisière avec une marge opérationnelle confortable en état stable. La RAM constitue le facteur limitant principal, avec un seuil critique identifié à 100 Mo disponibles. Les optimisations appliquées ont permis de libérer 109 Mo de RAM et 200 Mo d'espace disque sans dégradation des services actifs. L'architecture de sauvegarde distante chiffrée (`Backblaze B2` + `rclone`) s'intègre sans impact perceptible sur les performances, avec un RPO de 24 heures et un RTO inférieur à une heure.

Sur les attaques observées (chapitre 2) : Le dataset **Mogwainnova Research #001** confirme sans ambiguïté qu'un serveur exposé publiquement est ciblé de manière continue et automatisée. 1 819 987 événements ont été enregistrés en 39 jours depuis 4 331 adresses IP distinctes, avec une activité constante 24h/24 et 7j/7. Les attaques sont tactiquement simples : credentials par défaut, outils Go et libssh, rotation d'IPs. Cette simplicité n'est pas un signe de faiblesse des attaquants, mais d'une stratégie d'optimisation à grande échelle : tester des millions de cibles rapidement plutôt que d'attaquer une cible unique avec sophistication.

Sur les performances HTTP (chapitre 3) : Les tests de charge ont démontré une capacité réelle de 310–331 requêtes/seconde, soit 3 à 6 fois supérieure à l'hypothèse initiale conservatrice de 50–80 req/s. Le serveur se comporte de manière prévisible et contrôlée sous pression : il ralentit progressivement sans crash ni erreur, avec 100% de disponibilité sur 15 600 requêtes testées. Le facteur limitant principal identifié est l'établissement des connexions TLS, et non le CPU ou la RAM.

Sur le durcissement sécurité (chapitre 4) : Cinq mesures ont été implémentées pour un impact RAM total de 2 Mo : blocage du port 5355, durcissement de `/tmp` (`noexec`, `nosuid`, `nodev`), renforcement de `Fail2ban`, *rate limiting* `Nginx` et durcissement noyau via `sysctl`. Les solutions couramment recommandées (`ClamAV`, `Suricata`, `ModSecurity`) ont été délibérément exclues pour incompatibilité avec les ressources disponibles. Ces compromis sont documentés et assumés.

Réponse à la problématique

La réponse est affirmative, sous conditions. Un VPS 1 Go peut effectivement héberger une infrastructure multi-services exposée sur Internet, à condition de :

1. **Accepter et documenter les compromis matériels** : certaines solutions de sécurité de référence sont incompatibles avec 1 Go de RAM. Ce n'est pas un échec de configuration, c'est une contrainte structurelle à assumer explicitement.
2. **Prioriser les mesures à impact RAM nul ou minimal** : durcissement noyau (`sysctl`), règles firewall (UFW), configuration applicative (`Fail2ban`, `Nginx`) offrent une protection significative sans consommation mémoire notable.
3. **Dimensionner les services pour la cohabitation** : Nginx, Cowrie et rclone coexistent correctement à condition que la RAM disponible ne descende pas sous le seuil critique de 100 Mo.
4. **Surveiller activement** : en l'absence d'IDS réseau, la surveillance repose sur les logs `Fail2ban`, `Cowrie` et `Nginx`. Cette surveillance manuelle est suffisante pour un usage personnel ou éducatif, insuffisante pour un environnement de production critique.

Limites de l'étude

Plusieurs limites méthodologiques doivent être rappelées. Cette étude repose sur un échantillon unique : un seul VPS, une seule période d'observation de 39 jours, une seule localisation réseau. Les résultats empiriques sont contextuels et ne constituent pas une généralisation universelle. La géolocalisation des adresses IP sources est une approximation ; elle reflète la distribution des machines impliquées dans les attaques, pas l'origine réelle des attaquants. Enfin, les tests de charge ont été réalisés sans *keep-alive* HTTP, ce qui pénalise artificiellement les mesures de latence TLS ; les performances réelles en production seraient probablement supérieures.

Perspectives

Plusieurs axes d'approfondissement se dégagent naturellement de ce travail :

— **Extension du dataset** : prolonger la période d'observation au-delà de 39 jours et comparer avec d'autres honeypots déployés dans des régions réseau différentes permettrait de consolider les conclusions sur la distribution des attaques.
— **Analyse des commandes post-authentification** : `Cowrie` capture les commandes exécutées par les attaquants après connexion simulée. Cette dimension n'a pas été exploitée dans ce travail et constituerait une contribution complémentaire significative.
— **Tests de charge avec keep-alive** : un benchmark comparatif avec et sans *keep-alive* HTTP permettrait d'isoler précisément l'impact du handshake TLS et d'affiner les recommandations opérationnelles.
— **Montée en charge progressive** : tester le comportement du VPS avec les trois services actifs simultanément sous charge HTTP soutenue reste à réaliser ; les tests du chapitre 3 ont été conduits avec Cowrie et Fail2ban actifs mais sans charge SSH simultanée significative.

Remarque finale

Ce projet démontre qu'une infrastructure minimaliste exposée sur Internet est non seulement viable, mais instructive. La contrainte matérielle, souvent perçue comme un obstacle, s'est révélée être un cadre disciplinant.

Glossaire

[leftmargin=3cm, style=nextline]

Botnet Réseau de machines infectées (zombies) contrôlées à distance par un attaquant. Utilisé pour mener des attaques distribuées à grande échelle sans révéler l'origine réelle de l'opérateur.

Brute force Méthode d'attaque consistant à tester systématiquement toutes les combinaisons possibles de credentials jusqu'à trouver la bonne. Les outils automatisés permettent de tester des milliers de combinaisons par seconde.

CDN (*Content Delivery Network*) Réseau de serveurs distribués géographiquement permettant de servir le contenu web au plus proche de l'utilisateur. Utilisé également comme bouclier anti-DDoS en absorbant le trafic malveillant en amont du serveur d'origine.

Cowrie Honeypot SSH open source développé en Python. Simule un environnement shell pour capturer et enregistrer les interactions des attaquants sans exposer le système réel.

DDoS (*Distributed Denial of Service*) Attaque visant à rendre un service indisponible en le saturant de requêtes provenant de nombreuses sources simultanées. Difficile à contrer sans solution de filtrage amont (CDN, WAF).

Fail2ban Outil de détection d'intrusion basé sur l'analyse des logs. Bannit automatiquement les adresses IP ayant dépassé un seuil de tentatives d'authentification échouées.

Honeypot Leurre informatique délibérément exposé pour attirer et observer les attaquants. Permet de collecter des données sur les méthodes d'attaque sans risquer le système réel.

IDS (*Intrusion Detection System*) Système de détection d'intrusion analysant le trafic réseau en temps réel pour identifier des comportements suspects. Exemples : Suricata, Snort.

IoC (*Indicator of Compromise*) Élément technique permettant d'identifier une activité malveillante. Dans ce document, les signatures SSH (`SSH-2.0-Go`, `SSH-2.0-libssh`) constituent des IoCs fiables pour détecter le trafic automatisé.

IP spoofing Technique consistant à falsifier l'adresse IP source d'un paquet réseau pour masquer l'origine d'une attaque ou usurper l'identité d'un système légitime.

Keep-alive (HTTP) Mécanisme permettant de réutiliser une connexion TCP/TLS pour plusieurs requêtes HTTP successives, évitant de ré-établir la négociation TLS à chaque requête. Améliore significativement les performances en production.

LLMNR (*Link-Local Multicast Name Resolution*) Protocole de résolution de noms sur réseau local (port 5355). Inutile sur un serveur public et exploitable pour du *poisoning* en environnement interne.

Nginx Serveur web et reverse proxy haute performance. Utilisé dans ce projet pour servir le site web public et appliquer le rate limiting HTTP.

noexec / nosuid / nodev Options de montage Linux restreignant les permissions d'un système de fichiers. `noexec` : interdit l'exécution de binaires ; `nosuid` : bloque l'escalade de privilèges via SUID ; `nodev` : interdit les fichiers de périphériques.

Rate limiting Mécanisme limitant le nombre de requêtes acceptées par unité de temps et par source. Protège contre les abus automatisés sans impacter les utilisateurs légitimes.

rclone Outil de synchronisation et sauvegarde vers des services de stockage cloud. Utilisé ici pour sauvegarder les données critiques vers Backblaze B2 via protocole S3.

RPO (*Recovery Point Objective*) Durée maximale acceptable de perte de données en cas d'incident. Un RPO de 24 heures signifie qu'au pire, les données des dernières 24 heures pourraient être perdues.

RTO (*Recovery Time Objective*) Durée maximale acceptable pour restaurer un service après un incident. Un RTO d'une heure signifie que le service doit être opérationnel dans l'heure suivant l'incident.

SYN flood Attaque par déni de service exploitant le protocole TCP : l'attaquant envoie un grand nombre de paquets SYN sans compléter la poignée de main, saturant la table des connexions en attente. Contré par les SYN cookies (`net.ipv4.tcp_syncookies`).

sysctl Interface de configuration du noyau Linux permettant de modifier des paramètres système à chaud. Les paramètres de durcissement réseau sont persistés dans `/etc/sysctl.d/`.

TLS (*Transport Layer Security*) Protocole de chiffrement des communications réseau. TLS 1.3 est la version actuelle, utilisée ici pour chiffrer les connexions HTTPS (AES-256-GCM).

UFW (*Uncomplicated Firewall*) Interface simplifiée pour la gestion du pare-feu Linux (`iptables`). Permet de définir des règles de filtrage réseau persistantes avec une syntaxe accessible.

VPS (*Virtual Private Server*) Serveur virtuel privé : machine virtuelle hébergée sur un serveur physique partagé, avec des ressources dédiées

(CPU, RAM, disque) et une adresse IP fixe. Offre un accès root complet à un coût réduit.

WAF (*Web Application Firewall*) Pare-feu applicatif analysant le contenu des requêtes HTTP pour détecter et bloquer les attaques applicatives (injections SQL, XSS, etc.). Exemple : ModSecurity.